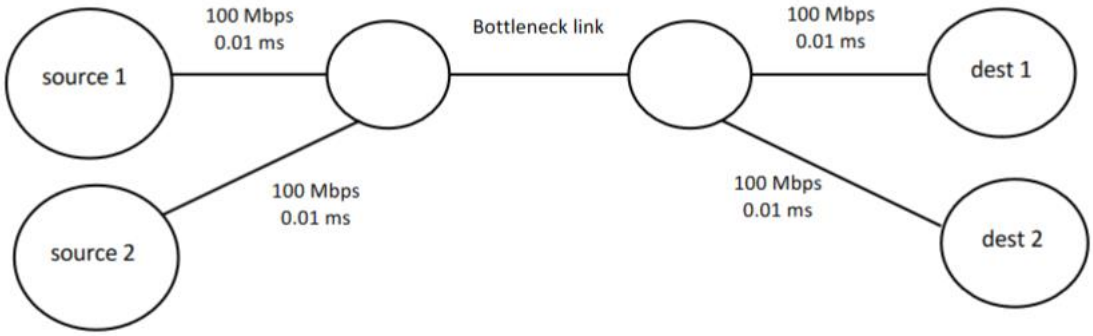
Félix Arnold

ECE 6110 Lab Assignment 2
Congestion Control

Part 1: Topology with a Bottleneck Link and Homogeneous Flows

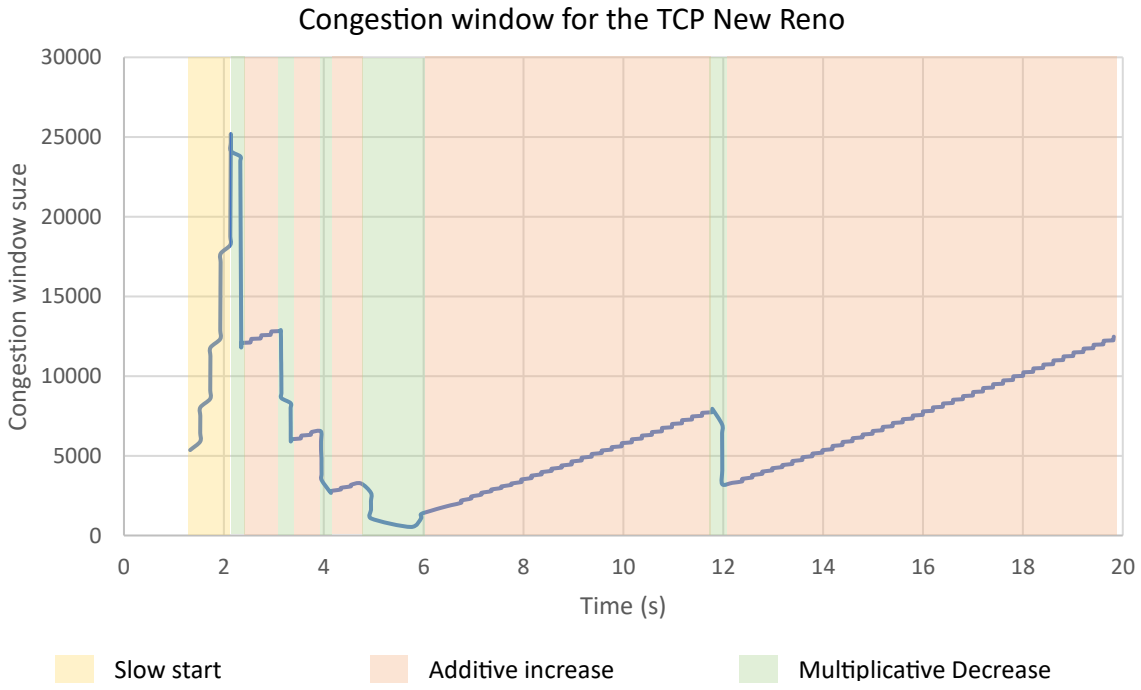
In this lab, using a ns3 simulation, we are going to first simulate the following topology:



Our goal is to observe and compare the New Reno and Cubic algorithms for congestion control with the Transmission Control Protocol (TCP).

Part 1a

For a bottleneck link data rate of 10 Mbps, delay of 100 msec, and error rate of 0.00001, and with two flows per source, we plot the congestion window size for both algorithm and for the first flow of the source 1.



Observing this graph, we can observe the 3 different kinds of behaviors of TCP New Reno algorithm:

- the slow start

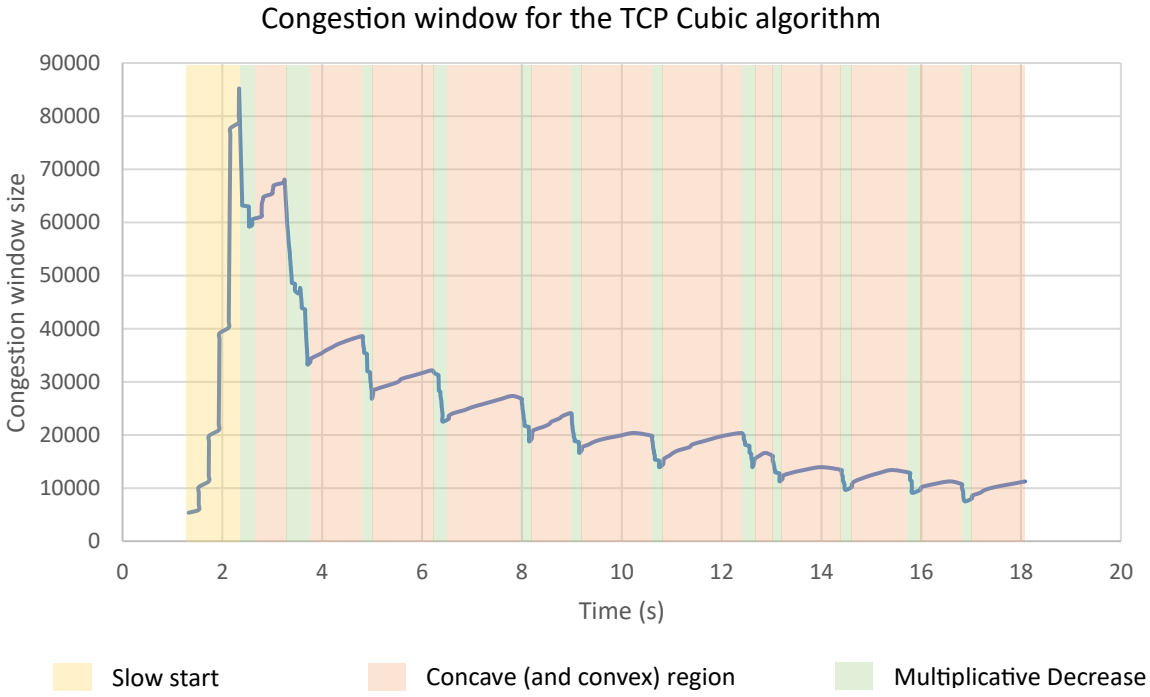
During this phase, the congestion window increases exponentially until it reaches a threshold value. We can observe it from the start of the simulation up to 2.14S where it reaches a size of 25192. I highlighted it in yellow in the graph.

- the additive increase

While no congestion is detected, the congestion window size is increased linearly (meaning by a fixed amount) for every received acknowledgment. We can observe it several times, for example between 6s and 11.9s. This behavior is highlighted in orange.

- the multiplicative decrease

When packet loss happens, it means that there is congestion. As such, the congestion window is reduced significantly as its size is divided by 2. We can notably see it between 11.9s and 12.2s. These phases are highlighted in green.



Again, we can observe with this plot different behaviors of the TCP Cubic algorithm:

- the slow start

Similarly, the TCP Cubic implementation use at first a slow start behavior up to a threshold value. We can observe it from the start of the simulation up to 2.33S where it reaches a size of 85224. I highlighted it in yellow in the graph.

- the concave region

When no congestion is detected, the congestion window size grows quickly until it reaches the size before the last congestion event. This behavior is highlighted in orange.

- the multiplicative decrease

  As previously, when packet loss happens, the congestion window size is divided by a fixed factor. We can see it between 6.2s and 6.5s. These phases are highlighted in green.

Another phase of TCP Cubic exists but is hardly visible in this plot. It corresponds to the convex region and takes place after a concave region. The algorithm probes for more bandwidth, at first slowly and then more and more rapidly. This behavior probably does not happen in our simulation because no flow end or start its transmission after the initial setup, so the general network conditions does not change. As such, congestion happens before the convex function can be executed to increase the window size.

When comparing both graphs, we can observe that the initial threshold of Cubic if much higher than the one of New Reno. Cubic is also more aggressive in its strategy as we see much more oscillation of the window size. It recovers much more quickly from congestion events, but these last ones happen more often. We also observe that the congestion window stays bigger with Cubic with a minimum value of 7504 (after the initial slow start) than with New Reno with which it reaches 536.

We can also study the goodputs of these two simulations. Thus, we get:

- for TCP New Reno:
  - flow from source 1 to sink 1: $32.283\ KB.s^{-1}$
  - flow from source 1 to sink 2: $29.732\ KB.s^{-1}$
  - flow from source 2 to sink 1: $31.266\ KB.s^{-1}$
  - flow from source 2 to sink 2: $24.381\ KB.s^{-1}$

  $=> Aggregate\ Goodputs = 117.659\ KB.s^{-1}$

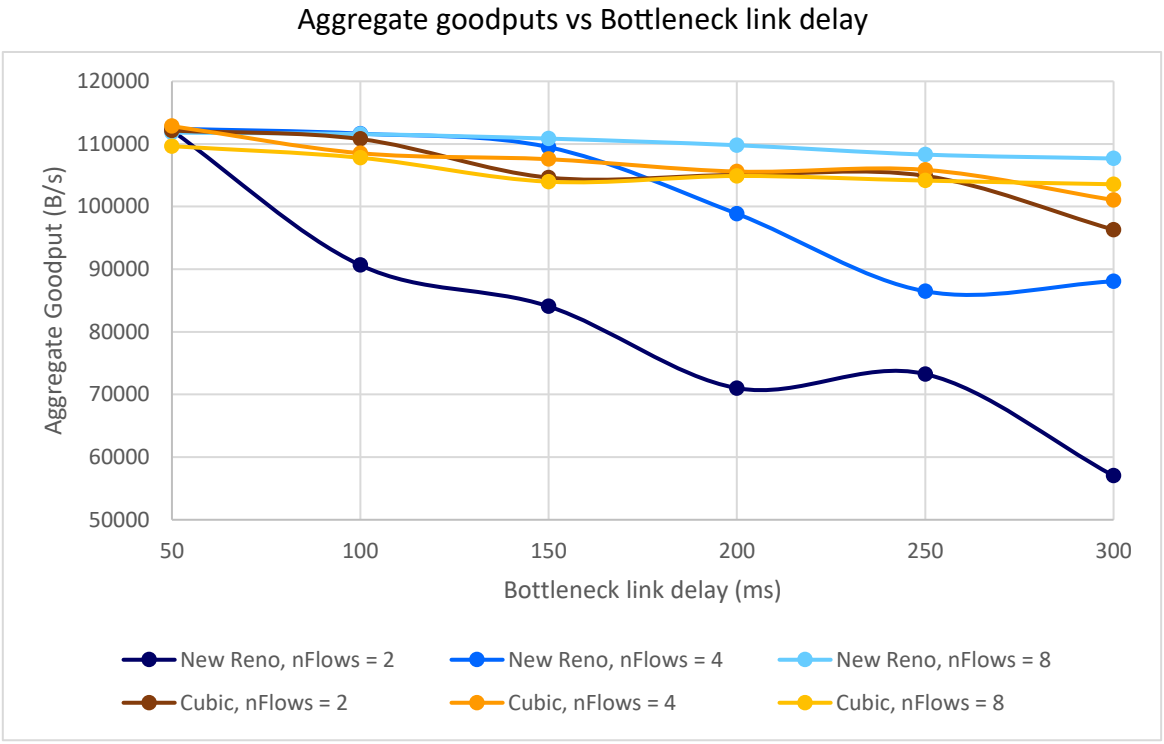
- for TCP Cubic:
  - flow from source 1 to sink 1: $125.221\ KB.s^{-1}$
  - flow from source 1 to sink 2: $104.345\ KB.s^{-1}$
  - flow from source 2 to sink 1: $58.369\ KB.s^{-1}$
  - flow from source 2 to sink 2: $44.339\ KB.s^{-1}$

  $=> Aggregate\ Goodputs = 332.273\ KB.s^{-1}$

Comparing their values, we can conclude that the TCP Cubic has much better performance than the New Reno. This is expected as the Cubic is a newer algorithm which was designed to increase the data transport speed. Thanks to its cubic window growth function, it is able to modify the window size much more quickly to better respond to the network. On the other hand, New Reno window size increases much more slowly which lose some useful bandwidth in exchange of more stability.

Part 1b

For a bottleneck link data rate of 1 Mbps and error rate of 0.00001, and for 2, 4, and 8 flows per source, we plot the aggregate goodput across all flows vs. bottleneck link delay for delays of 50 to 300 msec in steps of 50 and for both TCP CUBIC and TCP New Reno.



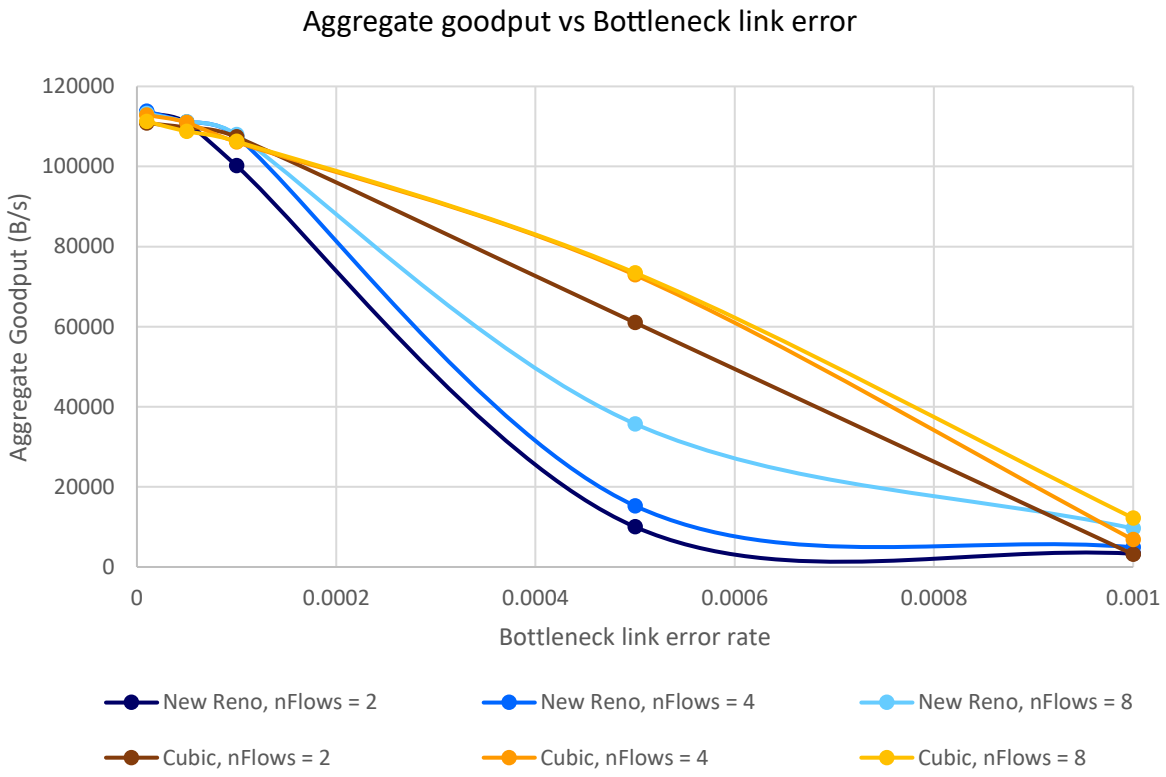
We observe, as explained previously, that for the same bottleneck link delay and number of flows, the Cubic aggregate goodputs are higher than the New Reno ones most of the time. Moreover, Cubic achieve a lower goodput variation than New Reno. It manages to keep the approximately the same goodput despite the increase in the number of flows.

If for a delay of 50ms all configurations have the same aggregate goodput roughly, as the delay increases, we observe for all curves a reduction of the aggregate goodput. However, it is much more significative for the New Reno simulations, notably for a low number of flows.

Nevertheless, for 8 flows per source, we observe that the New Reno algorithm performs better than the Cubic. This may happen because the TCP Cubic spend more time oscillating around the saturation point of the congestion window. On the other hand, the New Reno is more stable and reduce less frequently the window size which may explain this behavior.

Part 1c

For a bottleneck link data rate of 1 Mbps and delay of 1 msec, and for 2, 4, and 8 flows per source, we plot the aggregate goodput across all flows vs. bottleneck link error rate for error rates of 0.00001, 0.00005, 0.0001, 0.0005, and 0.001 for both TCP CUBIC and TCP New Reno.



For both protocols, the goodput decreases as the error rate increases since more packets needs to be retransmitted. Also, the goodput increases when we increase the number of flows, as previously.

We observe that TCP Cubic and New Reno achieve similar goodputs when the error rate is 0.00001 and when it is 0.001. However, for values in-between, TCP Cubic has much better goodputs. It seems this algorithm is more robust to increase in error rate, at least up to a certain point. Its goodput decrease is smoother while for New Reno it reaches its lower value much faster.